

Chapter 13: Reading the Command Line

[This section corresponds to K&R Sec. 5.10]

We've mentioned several times that a program is rarely useful if it does exactly the same thing every time you run it. Another way of giving a program some variable input to work on is by invoking it with *command line arguments*.

(We should probably admit that command line user interfaces are a bit old-fashioned, and currently somewhat out of favor. If you've used Unix or MS-DOS, you know what a command line is, but if your experience is confined to the Macintosh or Microsoft Windows or some other Graphical User Interface, you may never have seen a command line. In fact, if you're learning C on a Mac or under Windows, it can be tricky to give your program a command line at all. Think C for the Macintosh provides a way; I'm not sure about other compilers. If your compilation environment doesn't provide an easy way of simulating an old-fashioned command line, you may skip this chapter.)

C's model of the command line is that it consists of a sequence of words, typically separated by whitespace. Your main program can receive these words as an array of strings, one word per string. In fact, the C run-time startup code is always willing to pass you this array, and all you have to do to receive it is to declare `main` as accepting two parameters, like this:

```
int main(int argc, char *argv[])
{
    ...
}
```

When `main` is called, `argc` will be a count of the number of command-line arguments, and `argv` will be an array ("vector") of the arguments themselves. Since each word is a string which is represented as a pointer-to-char, `argv` is an array-of-pointers-to-char. Since we are not *defining* the `argv` array, but merely declaring a parameter which references an array somewhere else (namely, in `main`'s caller, the run-time startup code), we do not have to supply an array dimension for `argv`. (Actually, since functions never receive arrays as parameters in C, `argv` can also be thought of as a pointer-to-pointer-to-char, or `char **`. But multidimensional arrays and pointers to pointers can be confusing, and we haven't covered them, so we'll talk about `argv` as if it were an array.) (Also, there's nothing magic about the names `argc` and `argv`. You can give `main`'s two parameters any names you like, as long as they have the appropriate types. The names `argc` and `argv` are traditional.)

The first program to write when playing with `argc` and `argv` is one which simply prints its arguments:

```
#include <stdio.h>

main(int argc, char *argv[])
{
    int i;

    for(i = 0; i < argc; i++)
        printf("arg %d: %s\n", i, argv[i]);
    return 0;
}
```

(This program is essentially the Unix or MS-DOS `echo` command.)

If you run this program, you'll discover that the set of "words" making up the command line includes the command you typed to invoke your program (that is, the name of your program). In

other words, `argv[0]` typically points to the name of your program, and `argv[1]` is the first argument.

There are no hard-and-fast rules for how a program should interpret its command line. There is one set of conventions for Unix, another for MS-DOS, another for VMS. Typically you'll loop over the arguments, perhaps treating some as option flags and others as actual arguments (input files, etc.), interpreting or acting on each one. Since each argument is a string, you'll have to use `strcmp` or the like to match arguments against any patterns you might be looking for. Remember that `argc` contains the number of words on the command line, and that `argv[0]` is the command name, so if `argc` is 1, there are no arguments to inspect. (You'll never want to look at `argv[i]`, for `i >= argc`, because it will be a null or invalid pointer.)

As another example, also illustrating `fopen` and the file I/O techniques of the previous chapter, here is a program which copies one or more input files to its standard output. Since "standard output" is usually the screen by default, this is therefore a useful program for displaying files. (It's analogous to the obscurely-named Unix `cat` command, and to the MS-DOS `type` command.) You might also want to compare this program to the character-copying program of section 6.2.

```
#include <stdio.h>

main(int argc, char *argv[])
{
    int i;
    FILE *fp;
    int c;

    for(i = 1; i < argc; i++)
    {
        fp = fopen(argv[i], "r");
        if(fp == NULL)
        {
            fprintf(stderr, "cat: can't open %s\n", argv[i]);
            continue;
        }

        while((c = getc(fp)) != EOF)
            putchar(c);

        fclose(fp);
    }

    return 0;
}
```

As a historical note, the Unix `cat` program is so named because it can be used to concatenate two files together, like this:

```
cat a b > c
```

This illustrates why it's a good idea to print error messages to `stderr`, so that they don't get redirected. The "can't open file" message in this example also includes the name of the program as well as the name of the file.

Yet another piece of information which it's usually appropriate to include in error messages is the reason why the operation failed, if known. For operating system problems, such as inability to open a file, a code indicating the error is often stored in the global variable `errno`. The standard library function `strerror` will convert an `errno` value to a human-readable error message string. Therefore, an even more informative error message printout would be

```
fp = fopen(argv[i], "r");
if(fp == NULL)
    fprintf(stderr, "cat: can't open %s: %s\n",
            argv[i], strerror(errno));
```

If you use code like this, you can `#include <errno.h>` to get the declaration for `errno`, and `<string.h>` to get the declaration for `strerror()`.

Read sequentially: [prev](#) [next](#) [up](#) [top](#)

This page by [Steve Summit](#) // [Copyright](#) 1995-1997 // [mail feedback](#)